

MESERLIKE

작성자 : 안승현

목차

- Meserlike 소개
- 계획
 - 1주차 - 구조 설계
 - 2주차 - 맵 구현, 플레이어 생성
 - 3주차 - 몬스터 생성 및 AI 구현
 - 4주차 - 스킬 구현, 선택 로직 구현 및 사운드 추가
- 개발 후 느낀점 혹은 추후 개선 방향
- <https://youtu.be/zycwzQuWkkU> (YouTube 영상)

MESERLIKE 소개

- Cocos2d-x 3.17.2엔진 사용
- 스팀 뱀서라이크 모작
- 메이플스토리 리소스 추출 툴 WzComparerR2-KMS 사용
- C++ 언어를 사용하여 구현

1주차 – 구조 설계

```
class Managers
{
public:
    Managers();
    virtual ~Managers();

    static Managers& GetInstance();

    static SceneManager* Scene();

    static SoundManager* Sound();

    static InputManager* Input();

    static DataManager* Data();

    static UIManager* UI();

    static MapManager* Map();

    static GameManager* Game();

    void Clear();

private:
    static Managers instance;

    SceneManager* scene;
    SoundManager* sound;
    InputManager* input;
    DataManager* data;

    UIManager* ui;
    MapManager* map;
    GameManager* game;
};
```

```
Manager
├── Contents
│   ├── GameManager.cpp
│   ├── GameManager.h
│   ├── MapManager.cpp
│   └── MapManager.h
├── Core
│   ├── DataManager.cpp
│   ├── DataManager.h
│   ├── InputManager.cpp
│   ├── InputManager.h
│   ├── SceneManager.cpp
│   ├── SceneManager.h
│   ├── SoundManager.cpp
│   ├── SoundManager.h
│   ├── UIManager.cpp
│   └── UIManager.h
├── Managers.cpp
└── Managers.h
```

- Managers 싱글톤 클래스를 생성 하여 유일성, 접근성 확보
- DataManager, SoundManager 클래스 등 핵심 , 컨텐츠 클래스 접근을 용의하도록 하기 위해 Managers의 멤버변수로 생성

1주차 – 구조 설계

```
class Creature : public cocos2d::Node
{
    using Super = Node;
public:
    Creature();
    virtual ~Creature();

    virtual bool init() override;
    virtual void onEnter() override;
    virtual void onExit() override;

    virtual bool onContactEnter(cocos2d::PhysicsContact& contact);
    virtual void onContactExit(cocos2d::PhysicsContact& contact);

    CREATE_FUNC(Creature);

    Dir GetDir();
    void SetDir(Dir dir);
    void FlipDir();

    AnimateEx* GetAnimateEx();

    Stat* GetStat();
    BaseController* GetBaseController();
    SkillBook* GetSkillBook();
    void StopHitAction();
public:
    DelegateAction<> deadAction;
protected:
    Dir dir;

    cocos2d::Sprite* cSprite;

    std::shared_ptr<AnimateEx> animateEx;

    CreatureType _creatureType;

    std::shared_ptr<Stat> stat;
    std::shared_ptr<BaseController> controller;
    std::shared_ptr<SkillBook> skillBook;

    cocos2d::Action* invincibilityAction;

    cocos2d::EventListenerPhysicsContact* contact;
};
```

- 게임 콘텐츠의 핵심 클래스인 Creature 클래스 생성 및 정의 Player의 Monster의 부모인 클래스
- 매니저와 비슷한 기능을 하는 클래스 직접 기능을 구현하는 것 보단 여러기능으로 나뉜 클래스를 관리 및 실행하는 클래스
- 오브젝트 이미지, 방향, 애니메이션, 상태, 컨트롤러 등 다양한 기능을 관리한다.

1주차 – 구조 설계

```
class Utils
{
public:
    static float Clamp(float v, float a, float b){ ... }

    static cocos2d::Size Utils::GetVisibleSize(){ ... }

    // 해상도 크기 기반 위치 잡기
    static Size Utils::GetScreenPos(PivotPoint pivot){ ... }

    static cocos2d::Size Utils::GetScreenPos(PivotPoint pivot, float xRate, float yRate){ ... }

    // 부모 노드의 기반 위치 잡기
    static Size Utils::GetRelativeNodePosFromParent(PivotPoint pivot, Node* node){ ... }

    static cocos2d::Size Utils::GetRelativeNodePosFromParent(PivotPoint pivot, Node* node, float xRate, float yRate){ ... }

    // 해당 노드의 Pivot과 contentSize 기준으로 배치
    // 해당 노드의 Pivot이 Vec2(0.5, 0.5) 기준으로 작성 적용하는 노드의 Pivot은 상관
    static Size Utils::GetRelativeNodePos(PivotPoint pivot, Node* node){ ... }

    static cocos2d::Size Utils::GetRelativeNodePos(PivotPoint pivot, Node* node, float xRate, float yRate){ ... }

    static void RegisterBlockTouchOneByOne(Node* node){ ... }

    static Animation* SimpleCreateAnimation(std::string path, float sizeV, float sizeH){ ... }

    static Animation* SimpleCreateAnimation(AnimationData animData){ ... }

    static Dir XDir(float x){ ... }

    static cocos2d::Sprite* SetDefaultSprite(std::string key){ ... }

    static cocos2d::Animation* CreateAnimation(std::string spriteId, std::string stateId){ ... }

    static std::map<std::string, std::pair<AnimationData, cocos2d::Animation*>> CreateAnimationMap(){ ... }

    static Vec2 RotateVec2(const Vec2& vec, float angle){ ... }

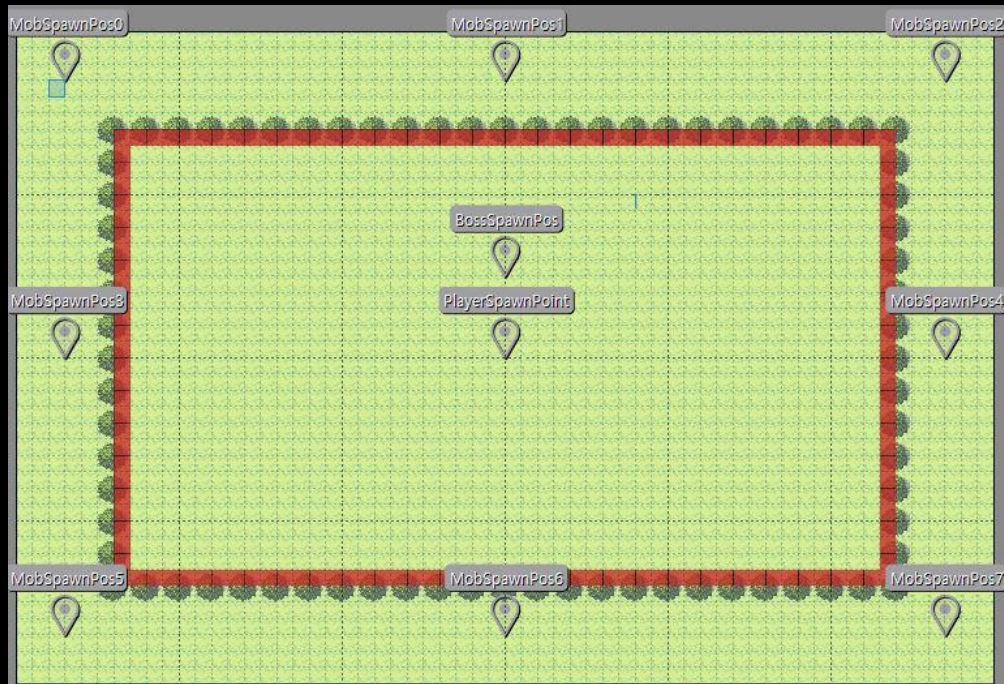
    static float GetDirAngle(Vec2& vec, Vec2& vec2){ ... }
};
```

```
class Bitmask
{
public:
    const static int Ignore = 0;
    const static int Ground = 1 << 0;
    const static int Wall = 1 << 1;
    const static int Object = 1 << 2;
    const static int Monster = 1 << 3;
    const static int Player = 1 << 4;
    const static int Invincibility = 1 << 5;
    const static int SkillObj = 1 << 6;
    const static int WorldItem = 1 << 7;
    const static int AllInvEx = INT32_MAX ^ (1 << 5);
    const static int All = INT32_MAX;
};
```

```
enum class MonsterType{ ... };
enum class PivotPoint{ ... };
enum class SceneType{ ... };
enum class SoundType{ ... };
enum class PopupType{ ... };
enum class Dir{ ... };
enum class State{ ... };
enum class CreatureType{ ... };
enum class SkillType{ ... };
enum class SkillId{ ... };
```

- Utils 정의 Cocos2d에 없는 기능을 추가 확장 (ex UI배치에 필요한 함수 구현, 두 벡터의 각도)
- BitMask 클래스로 충돌에 사용할 layer 값 정의
- Define 클래스로 대상의 값을 Enum값으로 정의

2주차 – 맵 구현, 플레이어 생성



- Tiled 프로그램으로 Meserlike 맵 제작 및 구현
- Tiled의 Object를 배치(몬스터 스폰, 플레이어 스폰 위치를 받아오기 위함)
- MapManager 클래스 생성 TileMap를 로드 및 관리

2주차 – 맵 구현, 플레이어 생성

```
class Player : public Creature, public IHitable
{
    using Super = Creature;
public:
    Player(int level);
    virtual ~Player();

    virtual bool init() override;
    virtual void onEnter() override;
    virtual void onExit() override;

    PlayerStat* GetPlayerStat();
    PlayerController* GetPlayerController();

    virtual void OnAttacked(float damage) override;
    virtual void OnAttacked(cocos2d::Node* other, float damage) override;

    virtual void update(float deltaTime) override;

    void Dash();

    CREATE_FUNC(Player);

    void SetCurDashCoolTime(float val);
    float GetCurDashCoolTime();

    void SetDashCoolTime(float val);
    float GetDashCoolTime();

    void CreateLevelUpEffect();

    DelegateAction<Float> dashStartDelegate;
    DelegateAction<Float, float> curDashDelegate;
    DelegateAction<Float, float> dashDelegate;
protected:
    float dashDuration;
    float curDashCooltime;
    float dashCooltime;
    bool isDash;
};
```

- Creature 클래스를 상속받은 Player 클래스 생성
- 플레이어만이 쓰는 함수 및 델리게이트 구현
- BaseController 클래스를 상속받는 PlayerController 클래스생성
 - PlayerController : 입력에 처리 및 상태 관리 애니메이션 재생 요청하는 클래스
- Stat 클래스를 상속받은 PlayerStat생성
 - PlayerStat: 해당 객체의 상태를 기록하는 클래스

2주차 – 맵 구현, 플레이어 생성

```
class InputManager
{
public:
    InputManager();
    virtual ~InputManager();

    void SetMoveInput(cocos2d::Point input);
    cocos2d::Point GetMoveInput();

    void Clear();
private:
    cocos2d::Point moveInput;
};
```

```
void SneakyJoystick::updateVelocity(Point point)
{
    // Calculate distance and angle from the center.
    float dx = point.x;
    float dy = point.y;
    float dSq = dx * dx + dy * dy;

    if(dSq <= deadRadiusSq)
    {
        velocity = Point::ZERO;
        degrees = 0.0f;
        stickPosition = point;
        Managers::Input()->SetMoveInput(velocity);
        return;
    }
}
```

```
void KeyboardInput::update(float dt)
{
    Super::update(dt);
    if (input)
    {
        Vec2 dir = Vec2(0, 0);

        if (_inputKeyCode & 1)
            dir += Vec2(0, 1);
        if (_inputKeyCode & 2)
            dir += Vec2(0, -1);
        if (_inputKeyCode & 4)
            dir += Vec2(-1, 0);
        if (_inputKeyCode & 8)
            dir += Vec2(1, 0);

        dir = dir.getNormalized();

        Managers::Input()->SetMoveInput(dir);
    }
}
```

- 플레이어 입력을 받을 PlayerInput 클래스 정의
- SneakyJoyStick는 UI의 JoyStick을 터치 한 이벤트의 값을 InputManger 클래스의 moveInput값을 수정한다.
- KeyboardInput은 EventListenerKeyboard 클래스가 사용자의 키보드 값을 감지해 update() 함수에서 입력이 감지되면 moveInput값을 수정한다.

2주차 – 맵 구현, 플레이어 생성

```
class AnimateEx
{
public:
    AnimateEx();
    AnimateEx(cocos2d::Node* owner, std::string name, cocos2d::Animation* defaultAnimate, bool roop = true);
    virtual ~AnimateEx();

    void Init(cocos2d::Node* owner, std::string name, cocos2d::Animation* defaultAnimate, bool roop = true, bool clear = true);
    void Claer();

    void SetOwner(cocos2d::Node* owner);

    // 애니메이션을 넘겨주면 애니메이트로 변환되어 관리된다.
    bool RegisterAnimation(std::string name, cocos2d::Animation* animation, bool roop = true);

    void PlayAnimation(std::string name);

    void StopAnimation();

private:
    // 애니메이션을 실행할 대상
    cocos2d::Node* _owner;

    // 실행 중인 액션
    cocos2d::Action* animAction;

    // 등록된 애니메이트
    std::map<std::string, std::pair<bool, cocos2d::Animation*>> animMap;
};
```

- AnimateEx 클래스 생성
- Cocos2d-x는 Animate를 따로 관리하는 클래스가 없어 이를 관리할 사용자 지정 클래스 생성
- 생성시 소유자 또는 기본 애니메이션을 설정, 그리고 다른 애니메이션을 등록하여 실행 할 수 있습니다.
- 저장된 애니메이션은 map으로 관리되며 key 값으로 애니메이션을 불러와 사용합니다.

3주차 – 몬스터 생성 및 AI 구현

```
class GameManager
{
public:
    GameManager();
    virtual ~GameManager();

    Monster* SpawnMonster(MonsterType type);
    Player* SpawnPlayer(int level);

    void DeSpawnCreature(Creature* creature);

    Player* GetPlayer();

    void Clear();

    int GetScore();
    void SetScore(int value);

    int GetRound();
    void SetRound(int value);

    DelegateAction<int> roundDelegate;

    std::vector<std::pair<int, SkillId>> GetRandomSkillId(int size = 3);
    void SelectSkill(int index, SkillId skillId, SkillInfo skillInfo, SkillBase* skillBase);
private:
    bool isGetFirstSkillId;

    std::vector<int> firstSkillVec;
    std::vector<std::pair<SkillId, int>> playerUseAbleSkill;
    std::set<Creature*> creatureSet;
    Player* player;

    int score;
    int round;
};
```

- 게임 정보를 관리할 GameManager 클래스 생성
- 플레이어 몬스터 생성 및 관리
- 현재 점수 및 라운드 관리
- 플레이어가 습득할 수 있는 스킬 리스트 관리
- 플레이어 변수 관리

3주차 – 몬스터 생성 및 AI 구현

- BuffSkillData.json
- CreatureSoundData.json
- CreatureSpriteData.json
- ETCSpriteData.json
- FirstSkillSetData.json
- MonsterInfoData.json
- MonsterStatData.json
- PlayerInfoData.json
- PlayerStatData.json
- PlayerUseAbleSkill.json
- PortionData.json
- RepeatSkillData.json
- RoundData.json
- SequenceSkillData.json
- SoundData.json

- PlayerStat, MonsterStat 등 게임에 사용할 데이터 json으로 정의

3주차 – 몬스터 생성 및 AI 구현

```
class SpawningPool
{
public:
    SpawningPool();
    virtual ~SpawningPool();

    void TrySpawn(int round);

    cocos2d::Action* spawnAction;

    void FindSpawnData();
    void WaveSpawnMonster();
    void SpawnMonster();
    void CheckNextRoundSpawn();
protected:
    int maxRepeatIndex;
    int waveIndex;
    int maxWaveIndex;

    float spawnInterval;
    float spawnWaveInterval;
    SpawnArrayData spawnArrData;
    std::vector<SpawnWaveArrayData> spawnWaveDatas;

    std::vector<cocos2d::Vec2> spawnVec;
    int monsterId;

    int maxSpawnCnt;
    int curSpawnCnt;

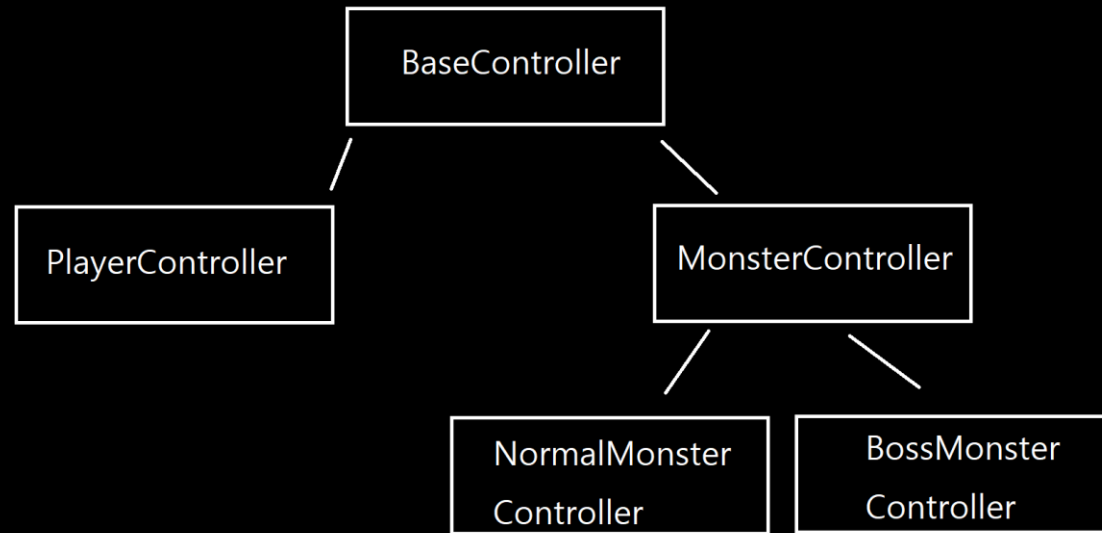
    int _callbackCnt;
    // 해당 라운드에 소환된 몹 수
    int _maxSpawnCount;
    // 몹 생성중인지
    bool _isSpawn;
};
```

RoundData.json

```
{
  "RoundSpawnArray": [
    {
      "round": 0,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 1,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 2,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 3,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 4,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 5,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    },
    {
      "round": 6,
      "spawnInterval": 0.5,
      "spawnData": [
        {
          "monsterId": 1,
          "x": 100,
          "y": 100,
          "angle": 0
        }
      ]
    }
  ]
}
```

- 몬스터 생성을 위한 SpawningPool 클래스 생성
- SpawningPool에 사용할 데이터를 Data 클래스에서 RoundSpawnData 구조체 정의
- 몬스터 생성 데이터를 기록한 RoundData.json 파일 생성

3주차 – 몬스터 생성 및 AI 구현



- BaseController를 상속받은 MonsterController 생성 그걸 상속 받은 NormalMonsterController, BossMonsterController 생성
- NormalMonsterController, BossMonsterController는 FSM으로 해당 상태에 따른 애니메이션을 재생 요청

3주차 – 몬스터 생성 및 AI 구현

```
class IHitable
{
public:
    virtual void OnAttacked(float damage) = 0;
    virtual void OnAttacked(cocos2d::Node* other, float damage) = 0;
};
```

```
class Player : public Creature, public IHitable
```

```
class NormalMonster : public Monster, public IHitable
```

```
class BossMonster : public Monster, public IHitable
```

- IHitable 인터페이스를 구현 하여 상속 받은 클래스만이 피격 처리를 하도록 할 수 있다
- Player, NormalMosnter, BossMonster 클래스만이 이 인터페이스를 상속받아 구현되어 있다.
- 이로 인해 IHitable 인터페이스만 있는지 체크하고 함수 호출만 하면 되기에 의존성 약화 및 능동적인 코드 작성이 가능하다.

4주차 - 스킬 구현, 선택 로직 구현 및 사운드 추가

```
class SkillBook
{
public:
    SkillBook(Creature* owner);
    virtual ~SkillBook();

    void AddSkillById(SkillId skillId, cocos2d::Vec2 position, Creature* parent, int level = 1);

    void StartNextSequenceSkill();

    void OnFinishedSequenceSkill();

    void StopSkills();

    SkillBase* GetSkill(SkillId skillId);

protected:
    int _sequenceIndex;
    bool _stopped;
    Creature* _owner;

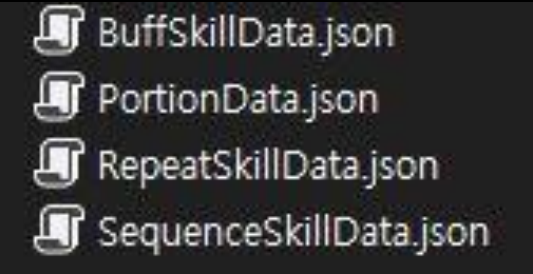
public:
    std::vector<SkillBase*> Skills;

    std::map<SkillId, SkillBase*> RepeatedSkills;
    std::vector<SequenceSkill*> SequenceSkills;
    std::map<SkillId, PassiveSkill*> buffSkills;
};
```

- 스킬을 관리할 SkillBook 정의
- 스킬의 조상 클래스 SkillBase 정의
 - RepeatSkill : 특정 시간마다 반복 하는 스킬
 - SequenceSkill : 순차적으로 사용하는 스킬
 - PortionSkill : 단발성 스킬
 - PassiveSkill : 지속 스킬
- SoundManger를 통해 UI 및 피격 효과에 따른 사운드 추가

4주차 - 스킬 구현, 선택 로직 구현 및 사운드 추가

- 스킬 데이터 정의
- 스킬의 id, 아이콘, 설명, 레벨, 스텟, 등 정의



BuffSkillData.json
PortionData.json
RepeatSkillData.json
SequenceSkillData.json

개발 후 느낀점 혹은 추후 개선 방향

- GUI없이 구현하다보니 UI 배치 또는 런타임에 값을 수정할 수 없어 많은 컴파일 및 테스트가 되어 개발 시간이 너무 오래 걸려서 힘들었다.
- 상용엔진의 소중함을 알게되었다.
- 하드 코딩이긴 했지만 툴 기능을 직접 만들어보는 것도 나쁘지 않고 좋은 경험이었습
니다.
- 사용자 정의 클래스는 cocos2d-x에서 관리되지 않아 개발자가 스스로 메모리를 관
리해야기 때문에 메모리에 신경써서 코딩함으로써 메모리 관리에 대한 경험 및 이해
가 생겼습니다.
- 모든 기능을 구현하다 보니 게임 구조에 대해 좀더 깊이있게 이해가 되었습니다.